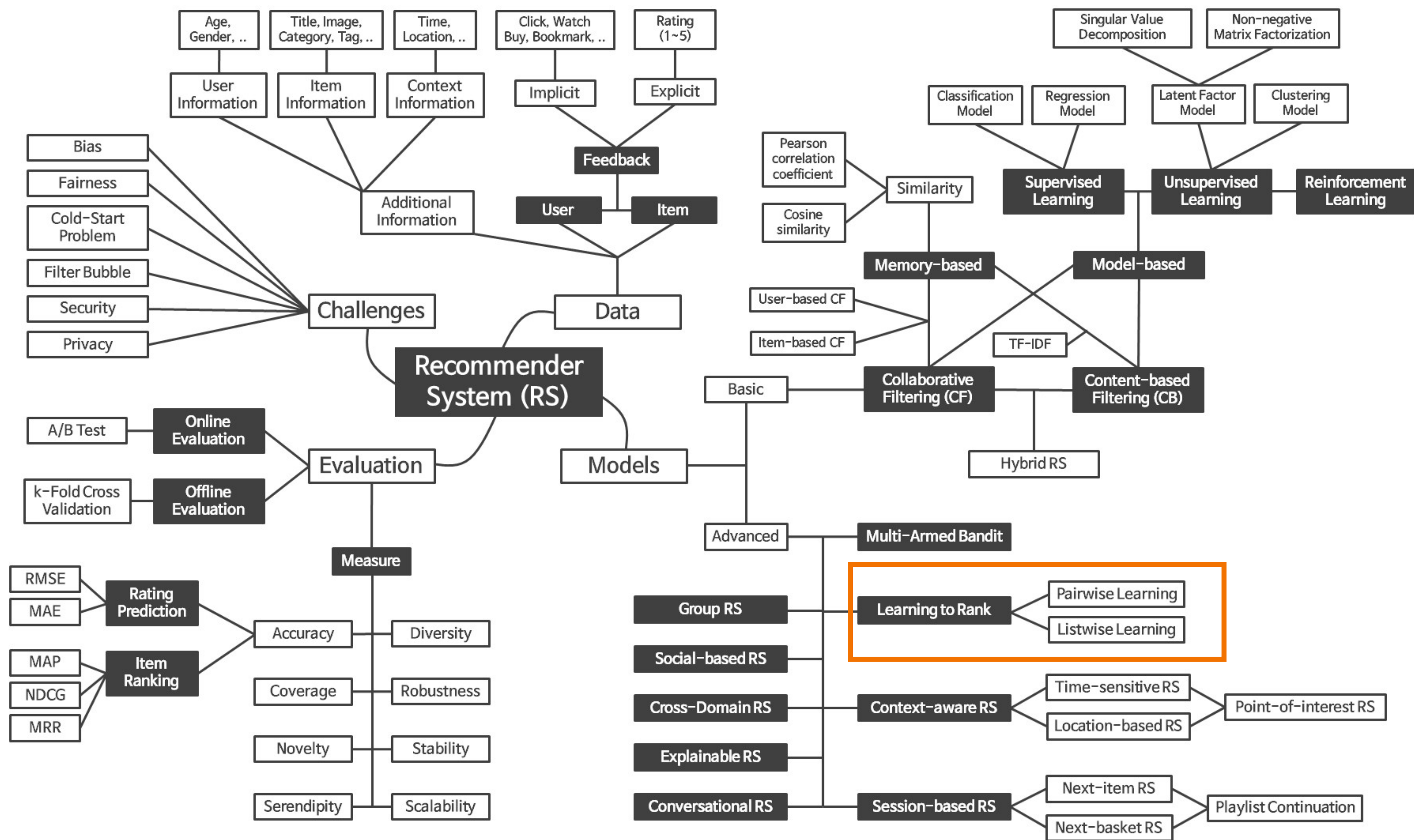


LTR, двухуровневые ранжирующие системы

Краснов Александр, 16.03.2026, AI masters



LTR, постановка задачи

LTR - Learning to rank

Пусть $X = \{x_1, \dots, x_n\}$ - обучающая выборка

отношение $i < j$ - x_j "лучше" x_i между документами из X

найти **ранжирующую функцию** $f: X \rightarrow \mathbb{R}$, которая восстанавливает порядок
 $i < j \Rightarrow f(x_i) < f(x_j)$

Простой пример: линейная функция $f(x, w) = \langle x, w \rangle$

LTR, примеры

I - коллекция документов

U - пользователи

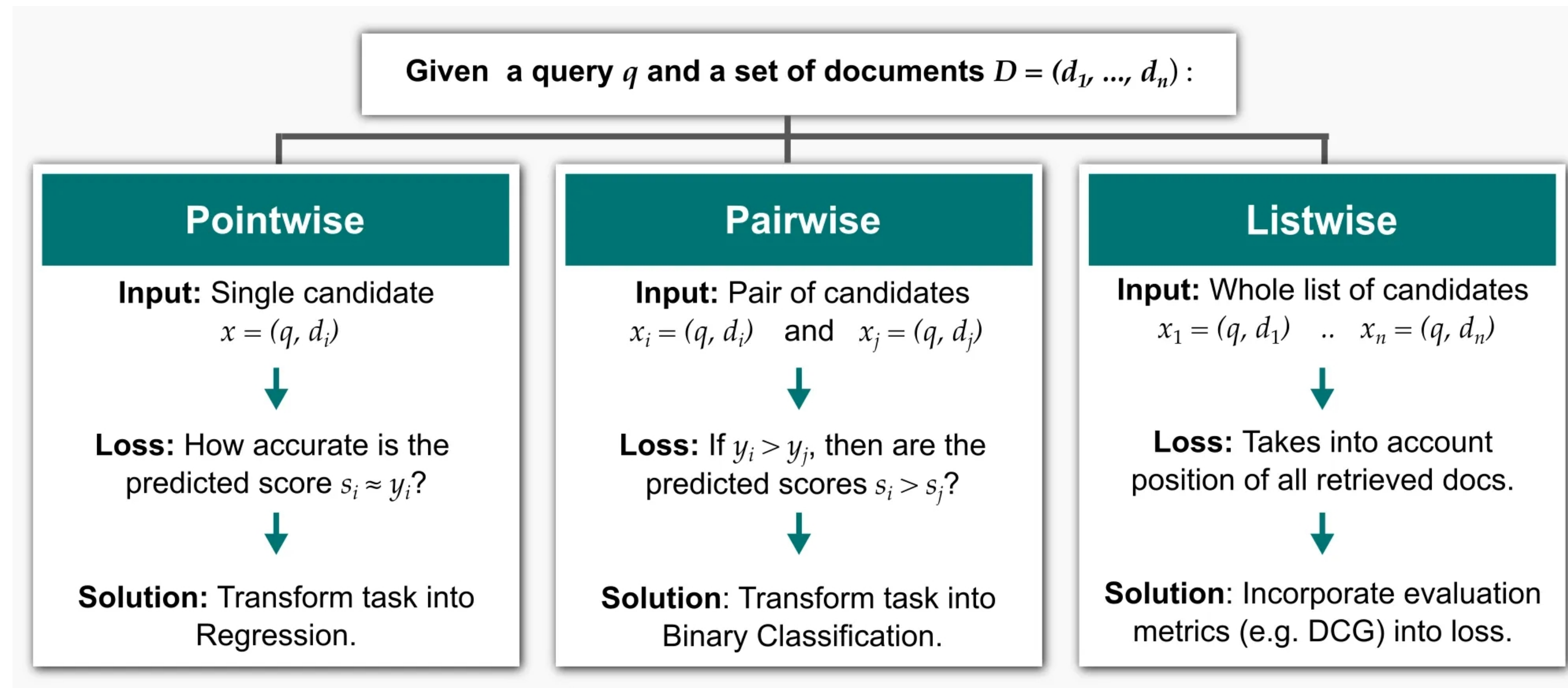
$X = U \times I$ - пары пользователь-документ

$$(u, i_k) \prec (u, i_m) \Rightarrow f(u, i_k) < f(u, i_m)$$

т.е. рекомендации пользователю u - список документов i , упорядоченный функцией $f(u, i)$

LTR, функции ранжирования

- point-wise (точечная оценка, например вероятность клика)
- pair-wise (попарное сравнение)
- list-wise (оценка списка, группы)



LtR, функции ранжирования

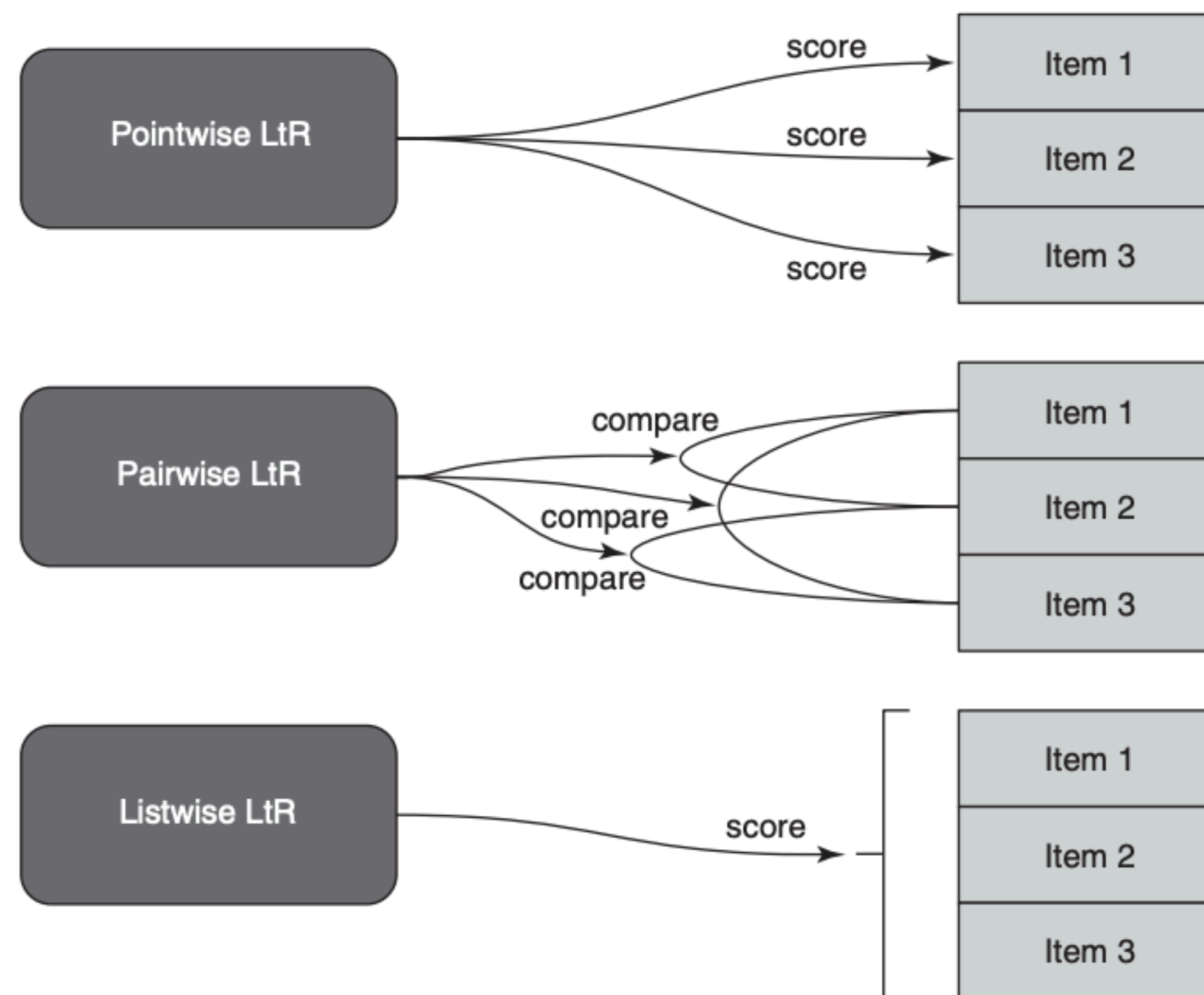
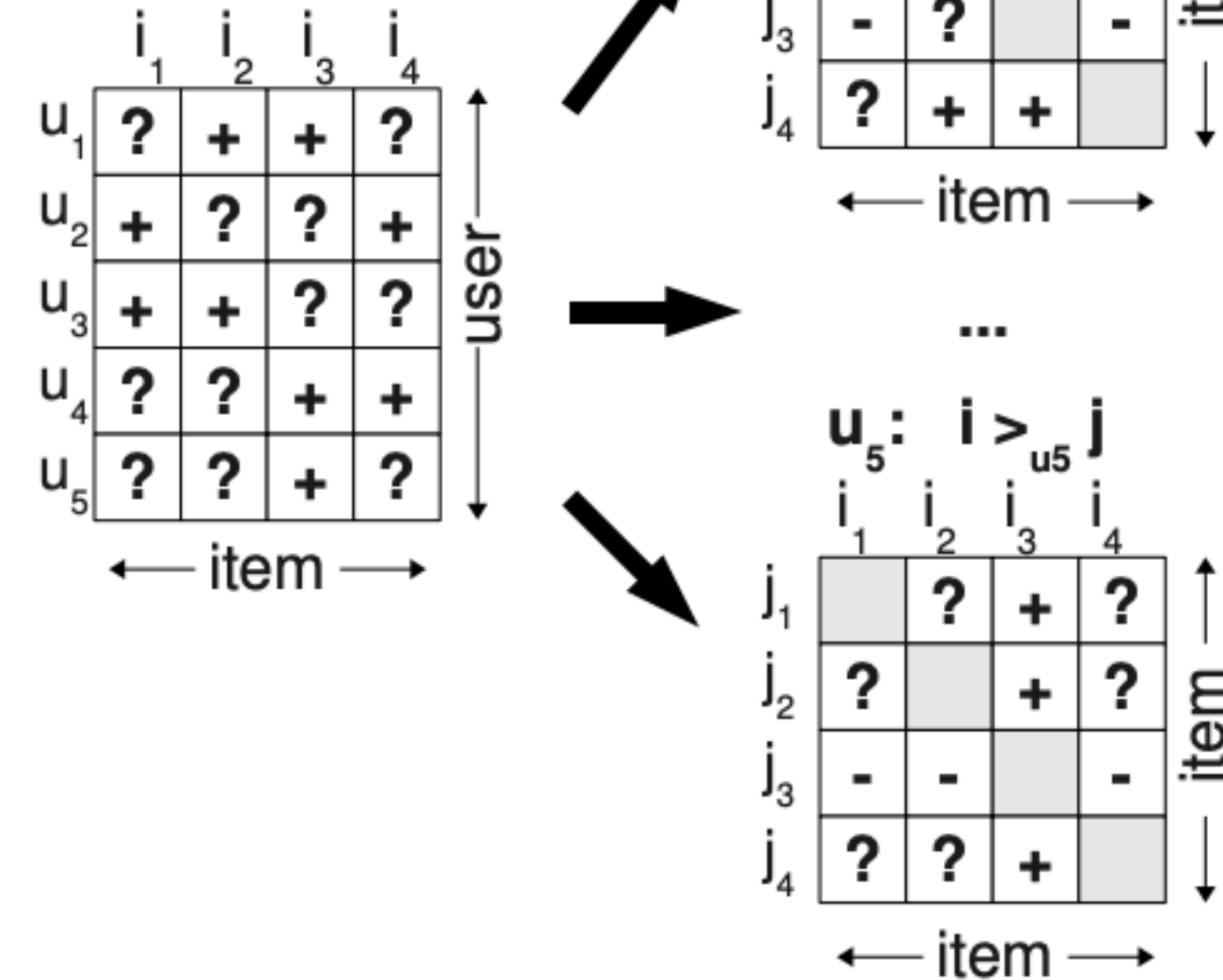
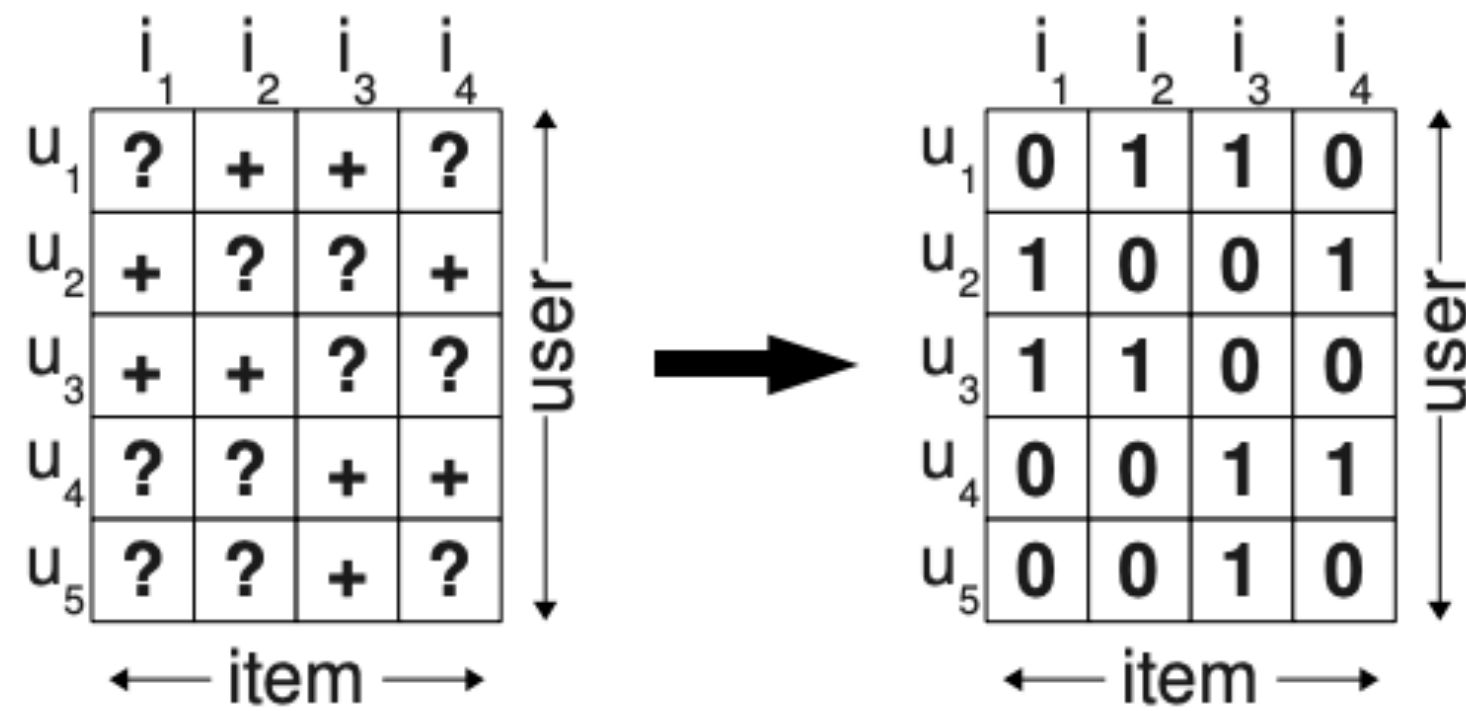


Figure 13.5 The three different subgroups of LtR algorithms: pointwise, pairwise, and listwise

LTR, BPR



$$D_S := \{(u, i, j) \mid i \in I_u^+ \wedge j \in I_u^+\}$$

$$L_{bpr} = -\frac{1}{D_S} \sum_{j=1}^{D_S} \log \sigma(r_i - r_j) + reg \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

BPR: Bayesian Personalized Ranking from Implicit Feedback

LTR, WARP

Интуиция алгоритма WARP - как и в BPR работаем с триплетами из D_S

1. Для (u, j) юзер-позитивный документ выбираем случайный негатив i среди всех остальных документов. Считаем предсказание - если порядок нарушился - обновляем градиент, иначе ищем другой негатив.
2. Чем быстрее находим негатив i , тем больше шаг градиента можем сделать.

LTR, WARP

Algorithm 1 K-OS algorithm for picking a positive item.

We are given a probability distribution P of drawing the i^{th} position in a list of size K . This defines the choice of loss function.

Pick a user u at random from the training set.

Pick $i = 1, \dots, K$ positive items $d_i \in \mathcal{D}_u$.

Compute $f_{d_i}(u)$ for each i .

Sort the scores by descending order, let $o(j)$ be the index into d that is in position j in the list.

Pick a position $k \in 1, \dots, K$ using the distribution P .

Perform a learning step using the positive item $d_{o(k)}$.

Algorithm 2 K-OS WARP loss

Initialize model parameters (mean 0, std. deviation $\frac{1}{\sqrt{m}}$).

repeat

 Pick a positive item d using Algorithm 1.

 Set $N = 0$.

repeat

 Pick a random item $\bar{d} \in \mathcal{D} \setminus \mathcal{D}_u$.

$N = N + 1$.

until $f_{\bar{d}}(u) > f_d(u) - 1$ or $N \geq |\mathcal{D} \setminus \mathcal{D}_u|$

if $f_{\bar{d}}(y) > f_d(u) - 1$ **then**

 Make a gradient step to minimize:

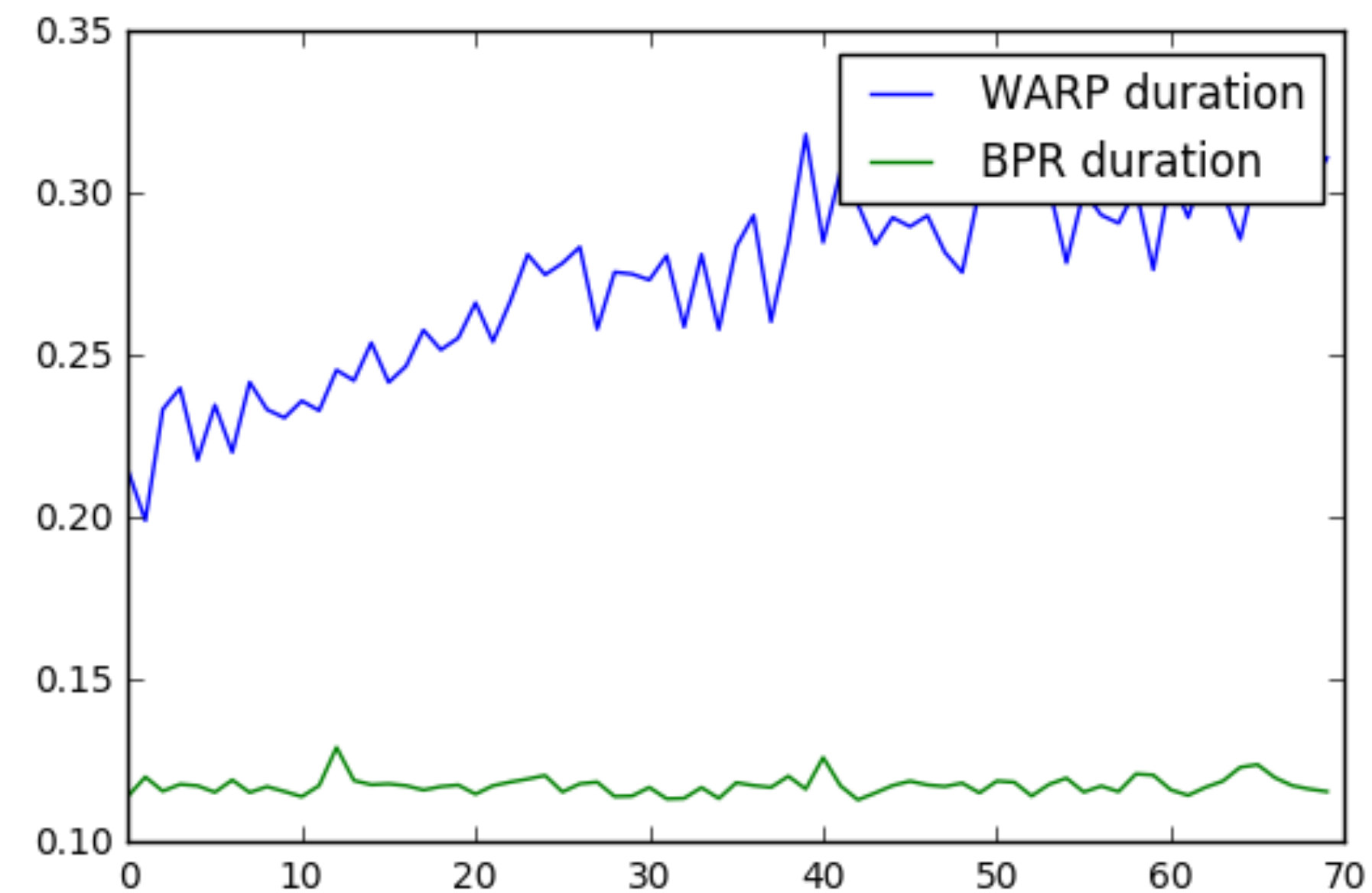
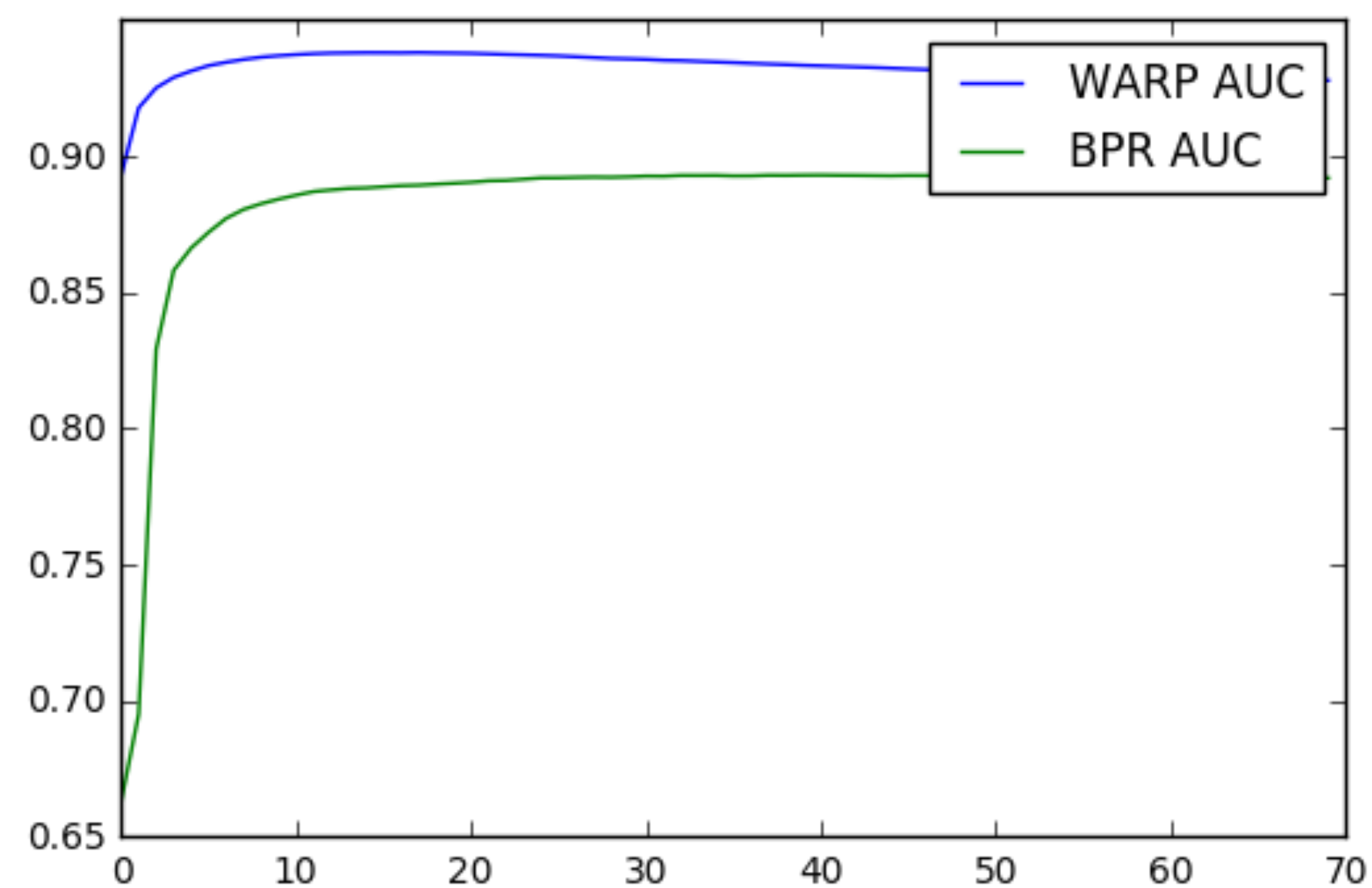
$$\Phi\left(\frac{|\mathcal{D} \setminus \mathcal{D}_u|}{N}\right) \max(0, 1 + f_{\bar{d}}(u) - f_d(u)).$$

 Project weights to enforce constraints, e.g. if $\|V_i\| > C$ then set $V_i \leftarrow (CV_i)/\|V_i\|$.

end if

until validation error does not improve.

LTR, BPR vs WARP



https://making.lyst.com/lightfm/docs/examples/warp_loss.html

LTR, RankNet

$$Pr(i \succ j) = P_{ij} \equiv \frac{1}{1 + e^{-(s_i - s_j)}}, s_i = f(q, d_i), s_j = f(q, d_j)$$

$$L = - \sum_{i \neq j} \frac{1}{2} (1 + S_{ij}) \log P_{ij} + \frac{1}{2} (1 - S_{ij}) \log (1 - P_{ij}), S_{ij} \in \{-1, 0, 1\}$$

LTR, RankNet

Факторизация градиента

$$\begin{aligned}\frac{\partial L}{\partial w_k} &= \sum_{\{i,j\}} \left[\frac{\partial L}{\partial s_i} \frac{\partial s_i}{\partial w_k} + \frac{\partial L}{\partial s_j} \frac{\partial s_j}{\partial w_k} \right] \\ &= \sum_i \frac{\partial s_i}{\partial w_k} \left(\sum_{\forall j < i} \frac{\partial L(s_i, s_j)}{\partial s_i} \right) + \sum_j \frac{\partial s_j}{\partial w_k} \left(\sum_{\forall i > j} \frac{\partial L(s_i, s_j)}{\partial s_j} \right) \\ \frac{\partial L(s_i, s_j)}{\partial s_i} &= - \frac{\partial L(s_i, s_j)}{\partial s_j} = \sigma \left[\frac{1}{2}(1 - S_{ij}) - \frac{1}{1 + e^{s_i - s_j}} \right] \\ \frac{\partial L}{\partial w_k} &= \sum_i \frac{\partial s_i}{\partial w_k} \left[\sum_{\forall j < i} \frac{\partial L(s_i, s_j)}{\partial s_i} + \sum_{\forall j < i} \frac{\partial L(s_j, s_i)}{\partial s_i} \right] \\ &= \sum_i \frac{\partial s_i}{\partial w_k} \left[\sum_{\forall j < i} \frac{\partial L(s_i, s_j)}{\partial s_i} - \sum_{\forall j > i} \frac{\partial L(s_j, s_i)}{\partial s_j} \right] = \sum_i \frac{\partial s_i}{\partial w_k} \lambda_i\end{aligned}$$

LTR, RankNet

Факторизация градиента

- Для каждого документа в данном запросе существует компонент вектора градиента, который мы обозначили как λ , которая вычисляется путем рассмотрения всех превосходящих и низшие документы по сравнению с ним
- Относительно худший документ будет толкать текущий документ вверх, а относительно лучший документ будет сдвинет его вниз.
- Во время обучения вместо обновления по каждой паре документов, мы можем обновлять по каждому запросу
- λ намного дешевле в вычислениях, весь процесс обучения может значительно ускориться

LTR, LambdaRank

- Проблема с RankNet в том, что оптимизируется число попарных ошибок, это не всегда то, что нужно.
- Как оптимизировать NDCG?

LTR, LambdaRank

$$\begin{aligned}\lambda_i &= \left[\sum_{\forall j < i} \frac{\partial L(s_i, s_j)}{\partial s_i} - \sum_{\forall j > i} \frac{\partial L(s_j, s_i)}{\partial s_j} \right] \\ &= \left[\sum_{\forall j < i} \lambda_{ij} - \sum_{\forall j > i} \lambda_{ij} \right] \\ \lambda_{ij} &\equiv \frac{\partial L(s_i, s_j)}{\partial s_i} \cdot |\Delta NDCG_{ij}| \end{aligned}$$

LTR, реализации

- BPR <https://github.com/Nemexur/revisit-bpr>
- WARP https://making.lyst.com/lightfm/docs/examples/warp_loss.html
- LambdaRank <https://github.com/haowei01/pytorch-examples/blob/master/ranking/LambdaRank.py>, <https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMRanker.html>
- YetiRank/YetiRankPairwise (посмотрим на семинаре) <https://catboost.ai/docs/en/concepts/loss-functions-ranking#YetiRank>

LTR, вопросы

Ранжирующие системы

Вводные

Мы хотим ...

- работать в реалтайме
- отвечать (отдавать список документов) быстро
- в момент запроса рассмотреть максимальное количество документов-кандидатов
- учесть максимальный объем доступной информации
- иметь доступную информацию максимально актуальной

Ранжирующие системы

Ограничения

- время ответа системы (10 - 1000 ms)
- большая база документов (1kk+)
- бизнес ограничения (не рекомендовать 18+ товары)
- ограничения по железу (CPU, RAM, GPU)

Многоуровневая система

retrieval

filtering

ranking

reranking

Retrieval

- кандидатогенерация/кандген/l1/retrieval
- отбор кандидатов из полного каталога документов
- сужаем воронку кандидатов до сотен/тысяч
- быстрые операции в момент запроса
 - достать по ключу из хэш-таблицы
 - ann/knn на gpu
 - быстрые модели расчета эмбединга
- оптимизируем метрику Recall@K

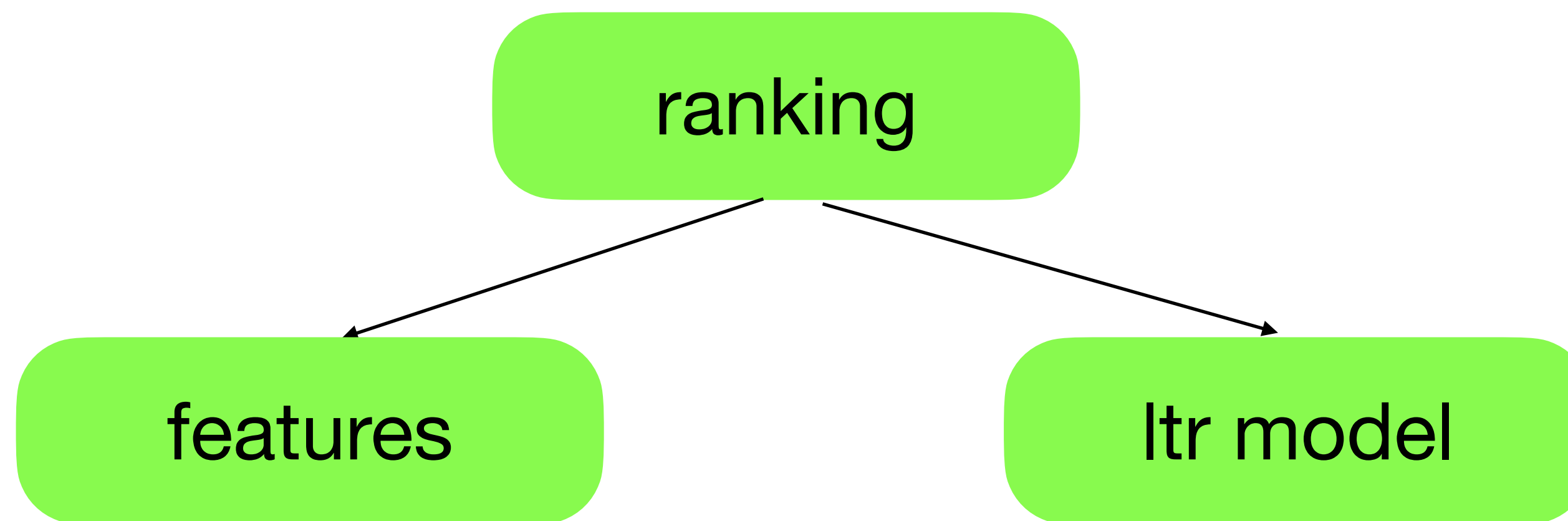
Filtering

- фильтрация по бизнес правилам (не показывать лекарства, 18+)
- фильтрация на доступность
- фильтрация при пагинации - фильтр товаров, которые были выше в ленте
- мало ml алгоритмов, много ручных правил

Ranking

- ранжирование/ranking/l2
- задача отобрать самых релевантных 10-1000 документов для re-ranking
- более сложные модели LTR (boosting, нейросетевые алгоритмы)
- расчет признаков $Q \times U \times I$
- NDCG, MAP

Ranking



- счетчики
- мета информация
- контекст* (время, локация, продукт, устройство)



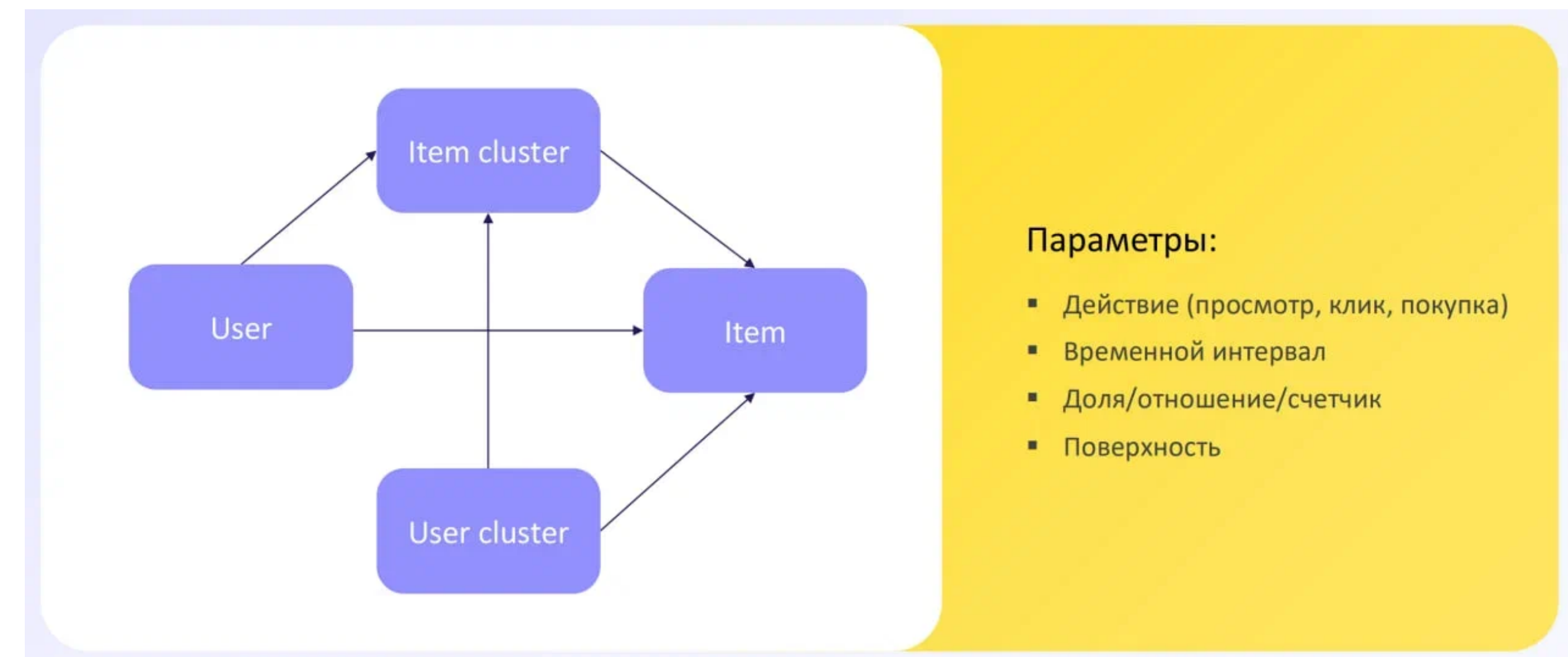
CatBoost



LightGBM

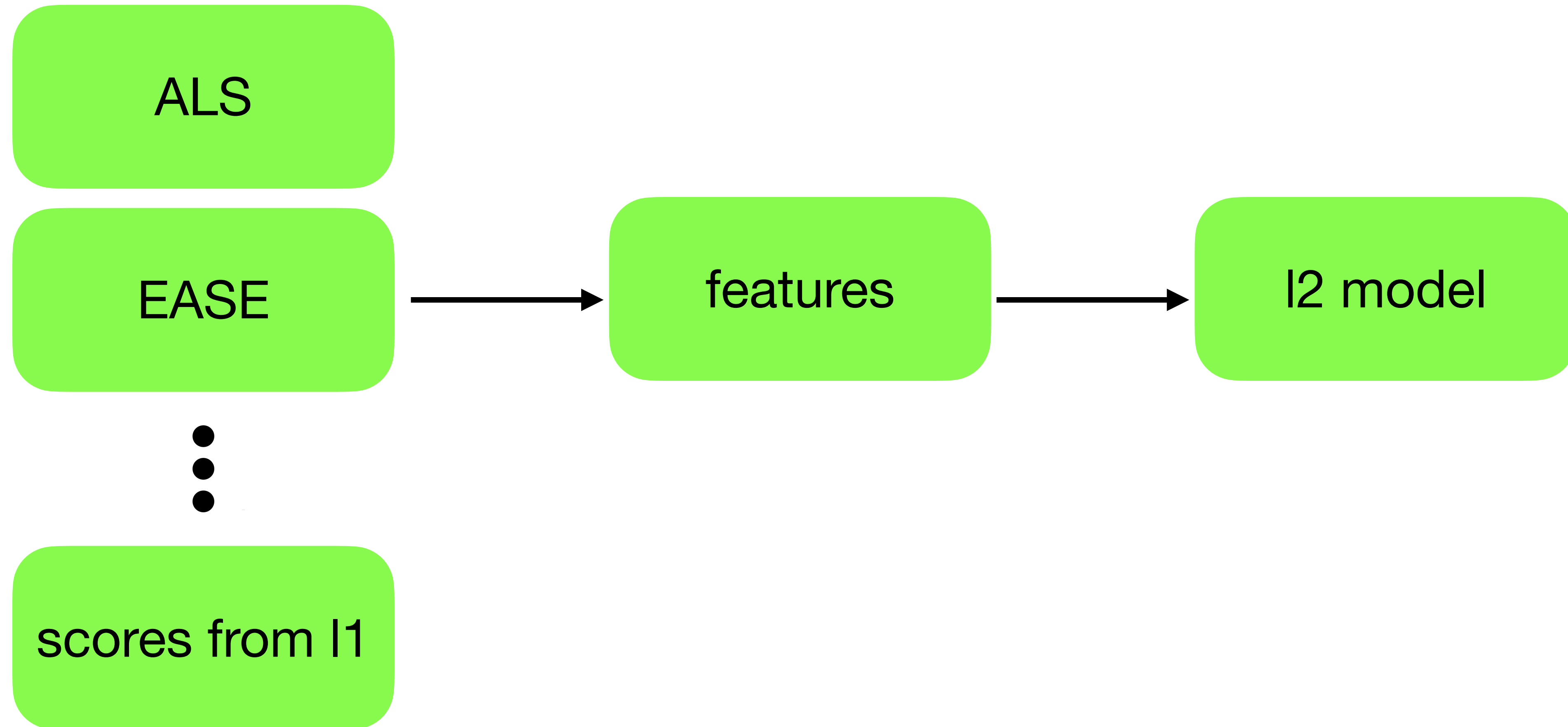
Ranking, Features

- I - конверсии, рейтинг, категориальная информация
- $U \times I$ - парные статистики, ценовые предпочтения, embedding dist
- $Q \times I$ - парные статистики, embedding dist
- $Q(I) \times I$ - item2item задача, embedding dist, совстречаемость в сессиях



Ranking, Features

Стекинг



Reranking

- reranking/I3
- задача сформировать финальный топ выдачи
- учесть разнообразие*, рекламные ставки (аукционы FPA, VCG)
- оценка кандидата с учетом места в финальной выдаче
- Diversity, Novelty, Serendipity

Reranking

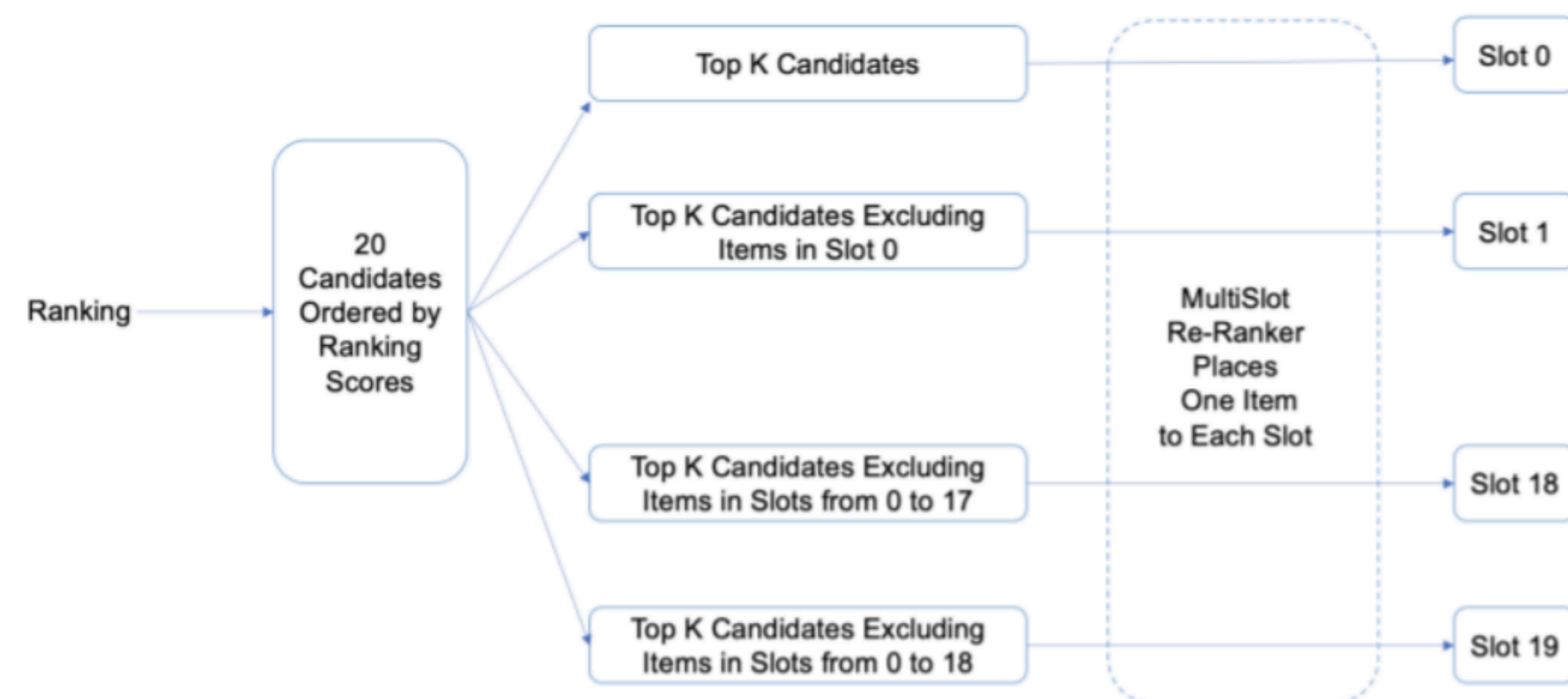
MultiSlot ReRanker

Algorithm 1 Sequential Greedy Algorithm

```

Require:  $i \geq 0$  and  $i < N$  ▷  $i$  is slot or position index,  $N$  is total number items.
Select item with top SPR score and place it to slot 0
 $i \leftarrow 1$  ▷ The slot 0 does not have previous slot
 $C \leftarrow \{0 \dots K - 1\}$  ▷ Initialize top  $K$  candidate set by second pass ranking scores
while  $i < N$  do
     $reranking\_score\_max \leftarrow -\infty$ 
     $k \leftarrow 0$  ▷ Select item  $k$  from Candidates
    for  $a \in C$  do
        Extract features from item  $a$ 
        Extract interaction features between item  $a$  and the previous slot  $i - 1$ 
        Calculate score by equation 1
        if  $score > reranking\_score\_max$  then
             $reranking\_score\_max \leftarrow score$ 
             $k \leftarrow a$  ▷ Pick the item with largest re-ranking score, i.e., greedy
        end if
    end for
    Remove item  $k$  from Candidate set  $C$  and place it to slot  $i$ 
    Pick the next top item in remaining list and add it to top  $K$  Candidate set  $C$ 
     $i \leftarrow i + 1$ 
end while

```



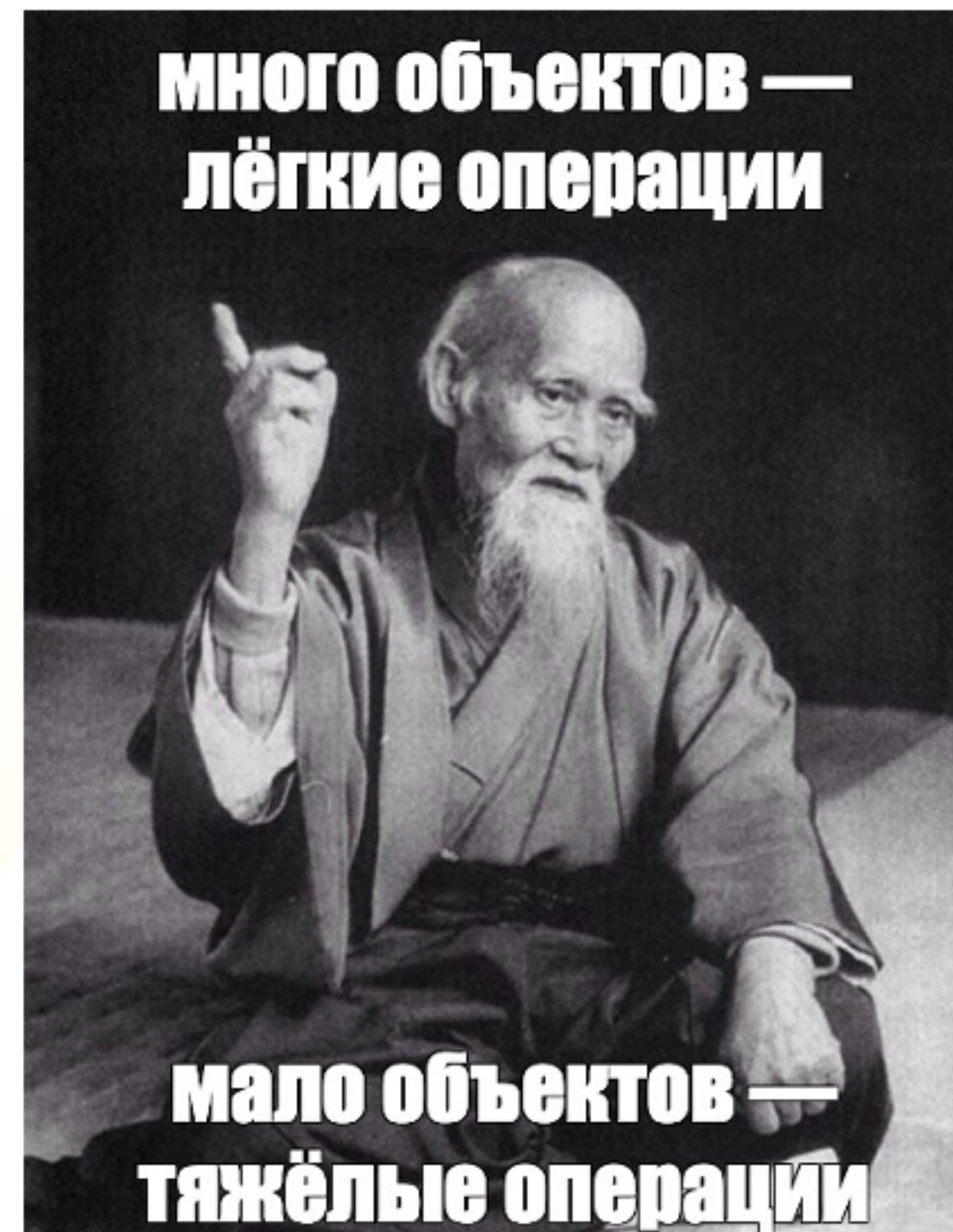
<https://arxiv.org/pdf/2401.06293>

Многоуровневая система

Базовый принцип

Воспользуемся мудростью сенсея
(он справа →)

- Нижние стадии → много объектов
→ самые лёгкие операции
- Отсекать как можно больше объектов
и как можно раньше
- Нижние стадии «глупые»
→ нельзя потерять важные объекты
- Чем дальше — тем более тяжёлые операции
можем выполнять

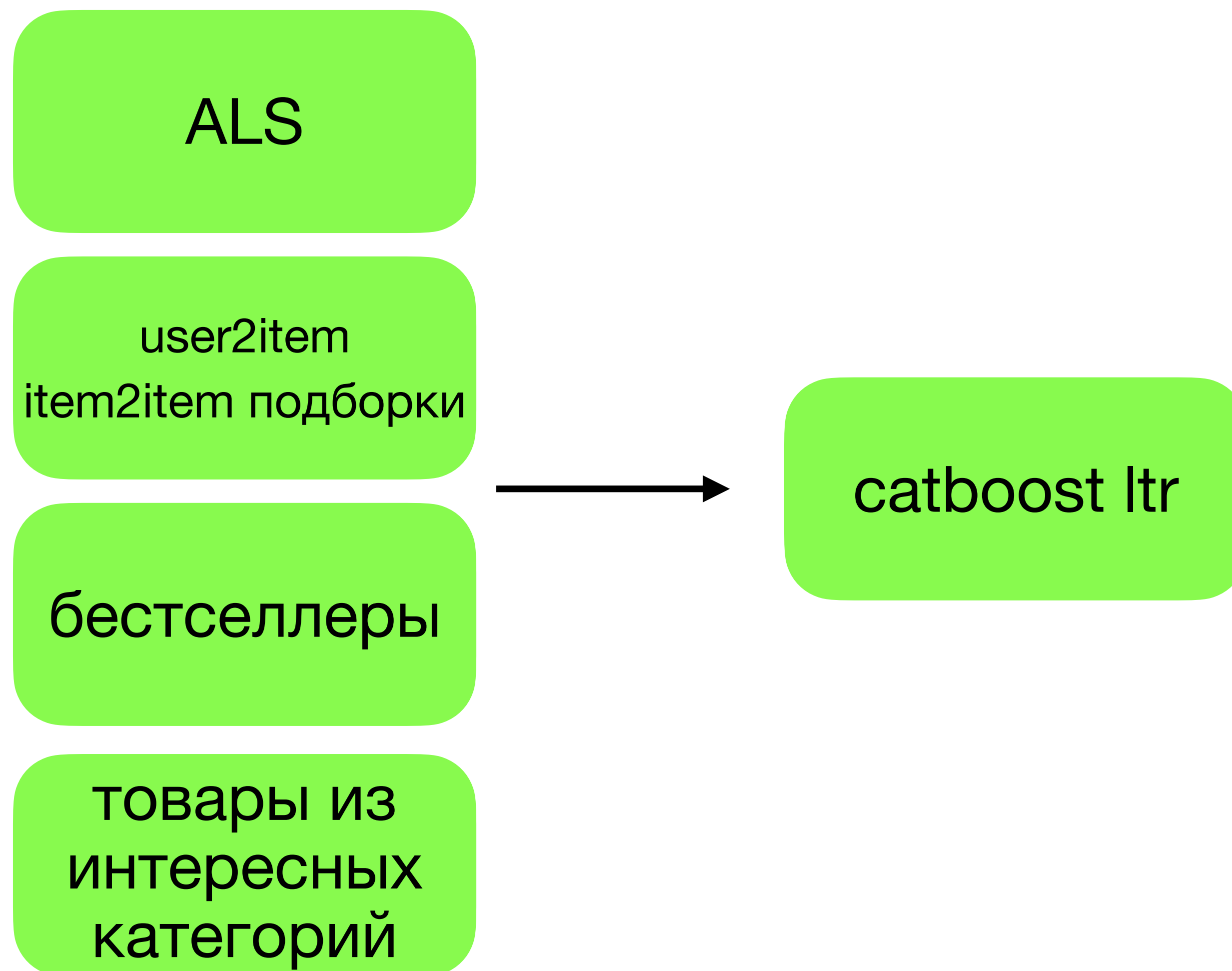


Retrieval vs Ranking

Task		Specification (Inputs / Outputs)
Ranking	$x_i S$	$\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{n_c-1}, a_{n_c-1}$
	$y_i S$	$a_0, \emptyset, a_1, \emptyset, \dots, a_{n_c-1}, \emptyset$
Retrieval	$x_i S$	$(\Phi_0, a_0), (\Phi_1, a_1), \dots, (\Phi_{n_c-1}, a_{n_c-1})$
	$y_i S$	$\Phi'_1, \Phi'_2, \dots, \Phi'_{n_c-1}, \emptyset$ $(\Phi'_i = \Phi_i \text{ if } a_i \text{ is positive, otherwise } \emptyset)$

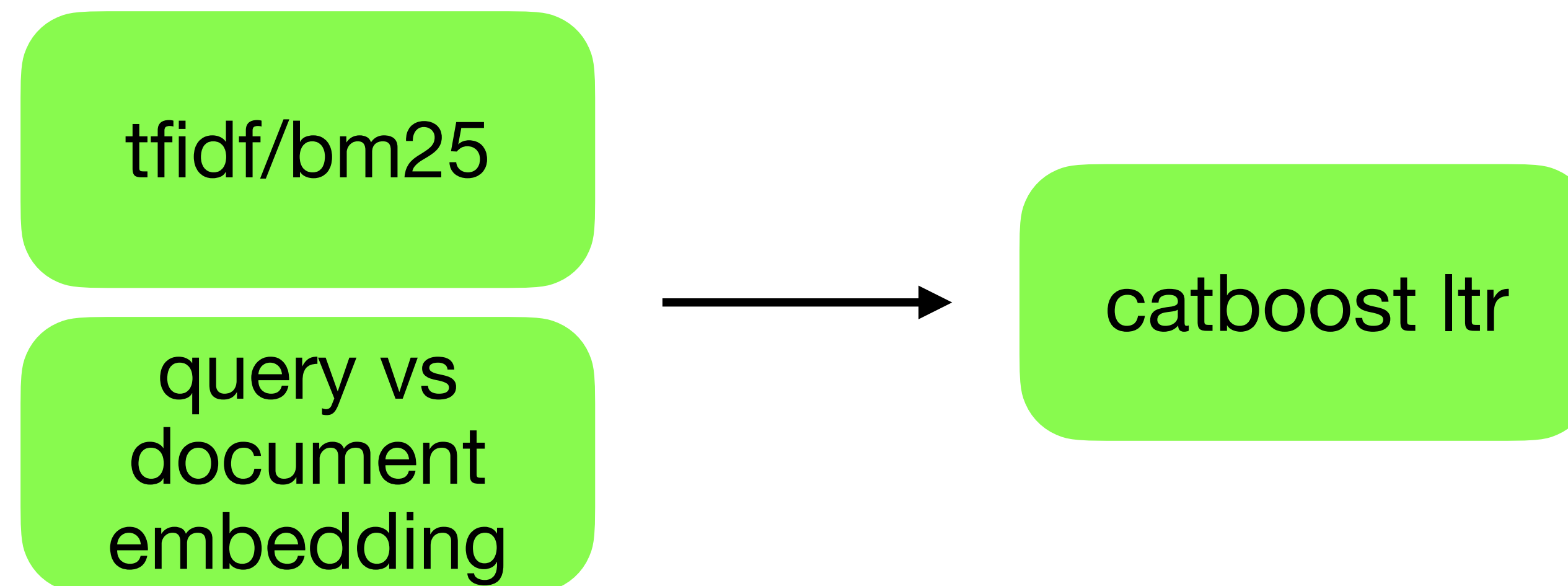
Многоуровневая система

Базовый пример из recsys



Многоуровневая система

Базовый пример из поиска



Retrieval, прочее

Несколько источников

Проблема: если источников больше 1, сколько кандидатов набирать из каждого?

Простое решение - задавать константами (ALS: 1000, I2I: 1000, TopPopular: 500) и проверять различные конфигурации через AB тесты.

Решение с ml - отдельной моделью предсказывать какой источник выбрать/пропорцию кандидатов из каждого источника

Retrieval, прочее

Легкое ранжирование

Иногда между этапами retrieval и ranking добавляют этап легкого ранжирования.

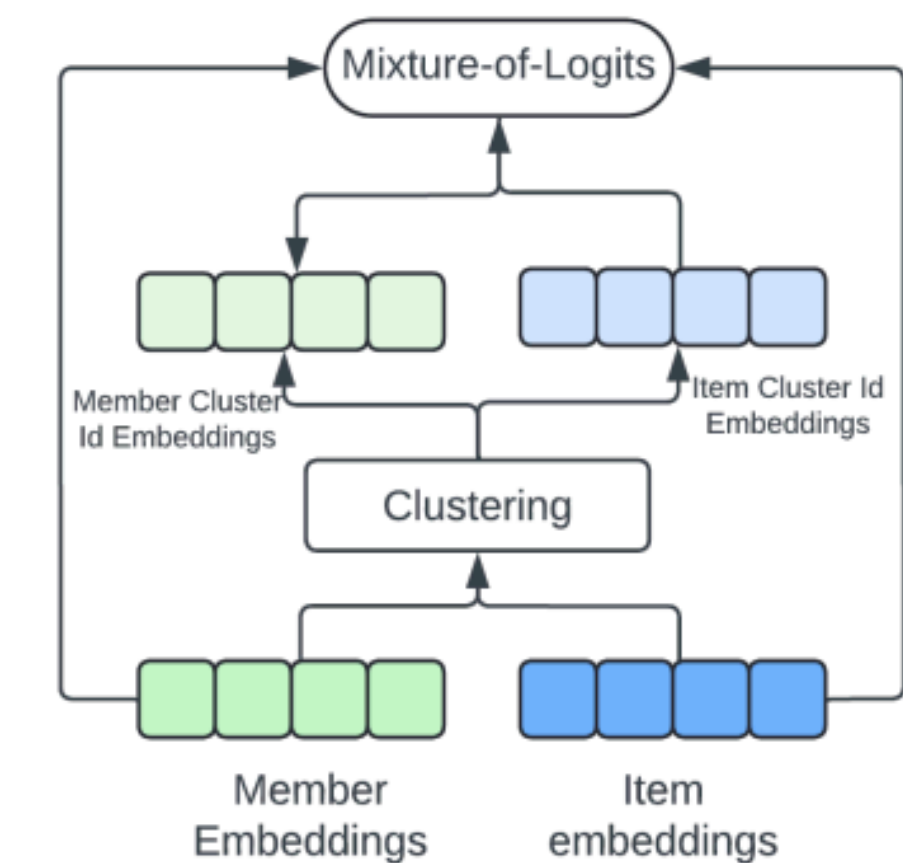
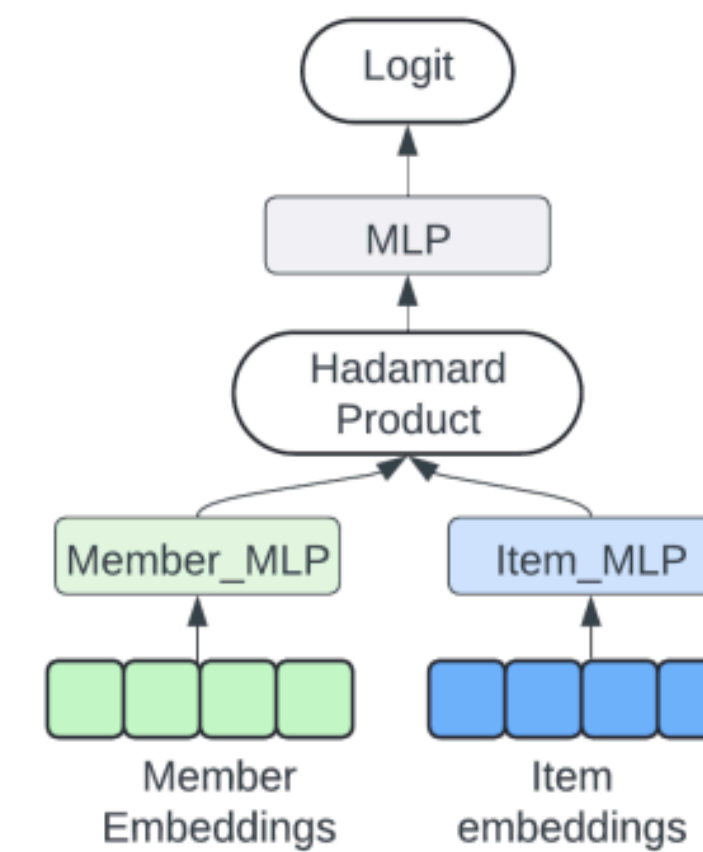
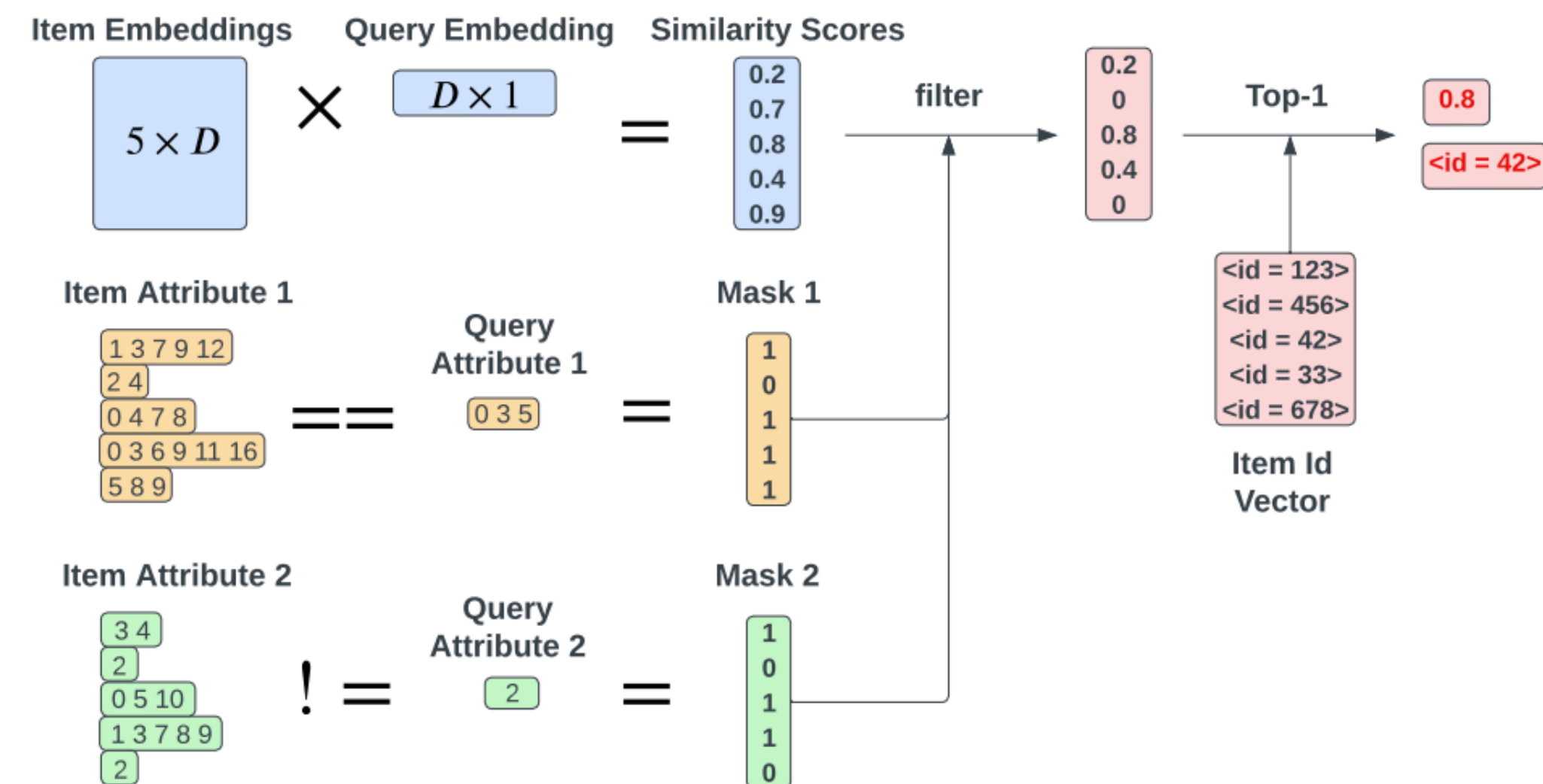
На этом этапе считаем меньше и легче признаки + более простая модель (линейная или бустинг с меньшим кол-вом деревьев)

Можно использовать дистилляцию (угадывать легкой моделью скоры тяжелой/финальный скор*)

<https://runtime.strm.yandex.ru/player/video/vplv5zetmiwwdym7bcv3?autoplay=0&mute=1>

Retrieval, прочее

ANN -> KNN



LiNR: Model Based Neural Retrieval on GPUs at LinkedIn

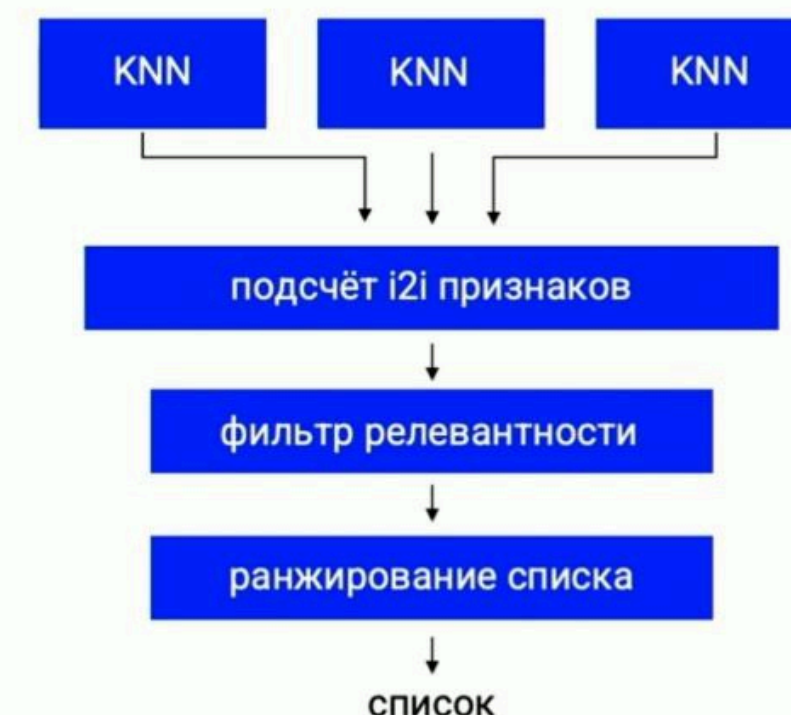
Retrieval, прочее

Item2item

- отдельный кандген – списки item2item, похожие документы
- можем использовать как для рекомендаций к самим документам
 - похожие по фото
 - рекомендации в корзине
- персональные рекомендации – похожие к позитивной истории
- легкие операции != легкий пайплайн расчета

<https://youtu.be/e-eZIRMgvIE?si=Ze2SSq8p17Ygye62>

Как считаются списки (~2 часа)



- Для 40M позитивов из истории готовим табличку с их эмбедингами

- Для каждого позитива насчитываем факторы для 100K кандидатов из приближенного KNN

- Применяем фильтр релевантности

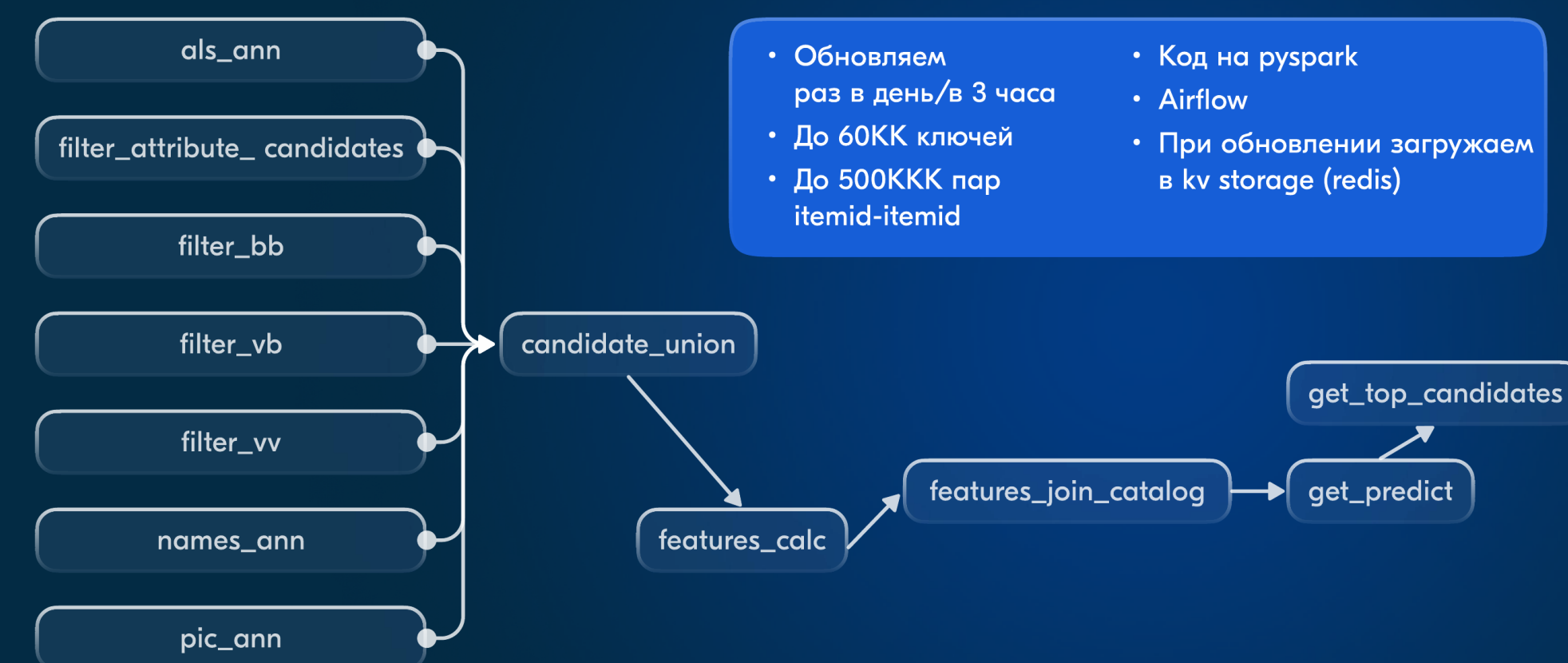
- *Неперсонально* ранжируем список

Одна Map операция

YfD Practical ML Conf

27

Как работает в офлайне



- Обновляем раз в день/в 3 часа
- До 60KK ключей
- До 500KKK пар itemid-itemid

- Код на pyspark
- Airflow
- При обновлении загружаем в kv storage (redis)

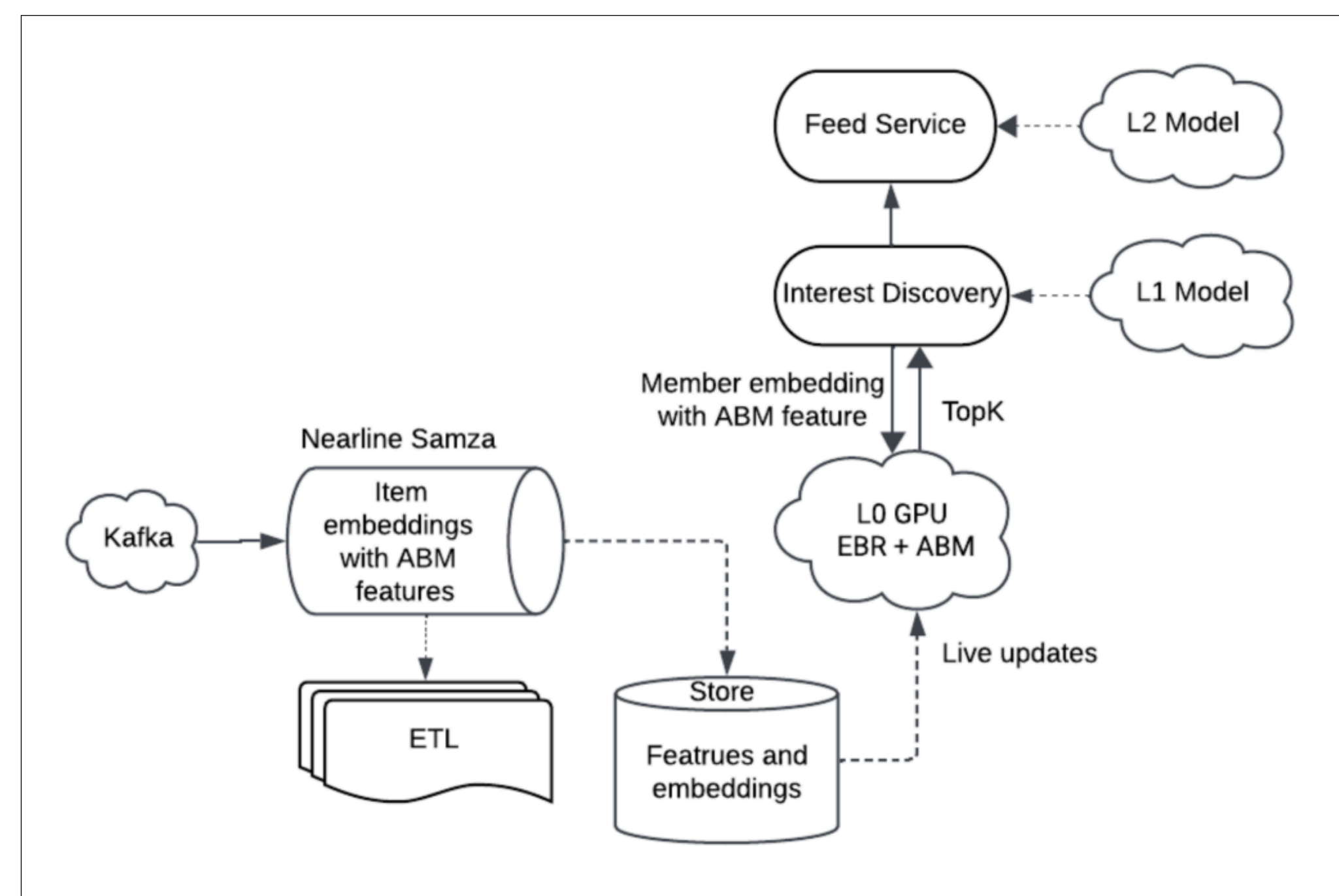
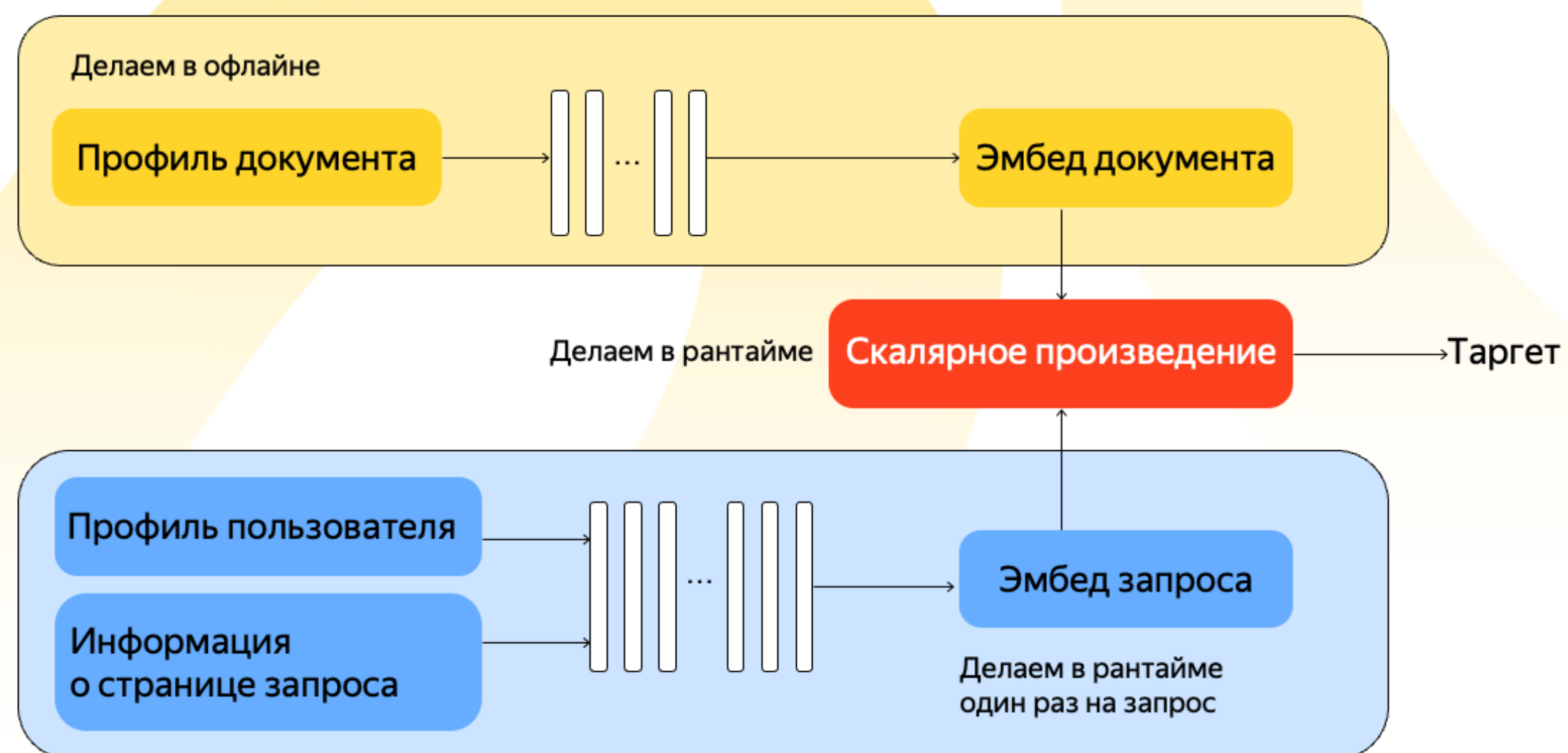
ozon{tech

16

Retrieval, прочее

Разделяй офлайн и онлайн

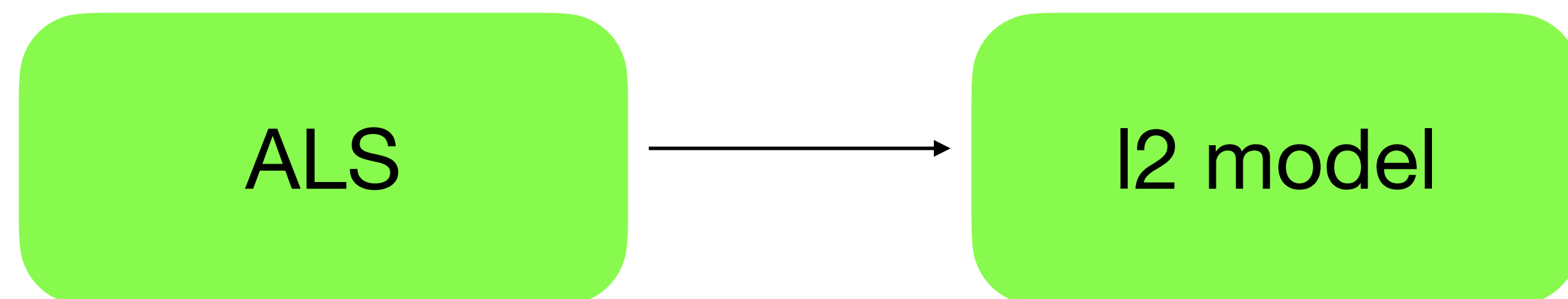
Как быстро применять нейросети в рантайме



Многоуровневая система

Как это все учить?

- offline timesplit



- на логах сервиса



Многоуровневая система

Логи, level 1

- интеракции user-item, query-item
- одна строка – одно действие
- ПОЗИТИВЫ И НЕГАТИВЫ

user_id	timestamp	action_type	widget_name	search_query	product_id	page_product_id
i32	datetime[ns]	str	str	str	f64	f64
5191333	2024-03-20 11:04:06	"view"	"search_catalog_listing"	null	9.23442542e8	null
3741045	2024-06-27 18:39:09	"view"	"search_catalog_listing"	"печенье"	6.18168589e8	null
5736030	2024-03-08 14:50:14	"view"	"search_catalog_listing"	null	1.2156e9	null
2173455	2024-04-03 12:57:27	"view"	"recoms_fresh_personal"	null	1.46227155e8	null
9569113	2024-07-26 01:49:36	"view"	"search_catalog_listing"	null	1.0800e9	null
...	...	...	...	...	...	...
3795880	2024-04-09 18:01:25	"view"	"recoms_fresh_personal"	null	5.01074615e8	null
6916747	2024-04-26 14:46:01	"view"	"search_catalog_listing"	null	1.43294378e8	null
2346229	2024-06-28 17:16:12	"view"	"recoms_fresh_personal"	null	4.28674301e8	null
441072	2024-06-08 05:26:42	"view"	"search_catalog_listing"	null	7.05609233e8	null
2137645	2024-03-05 18:35:39	"view"	"search_catalog_listing"	null	2.43970371e8	null

Многоуровневая система

Логи, level 2

```
message TRecords {
  reserved 14;

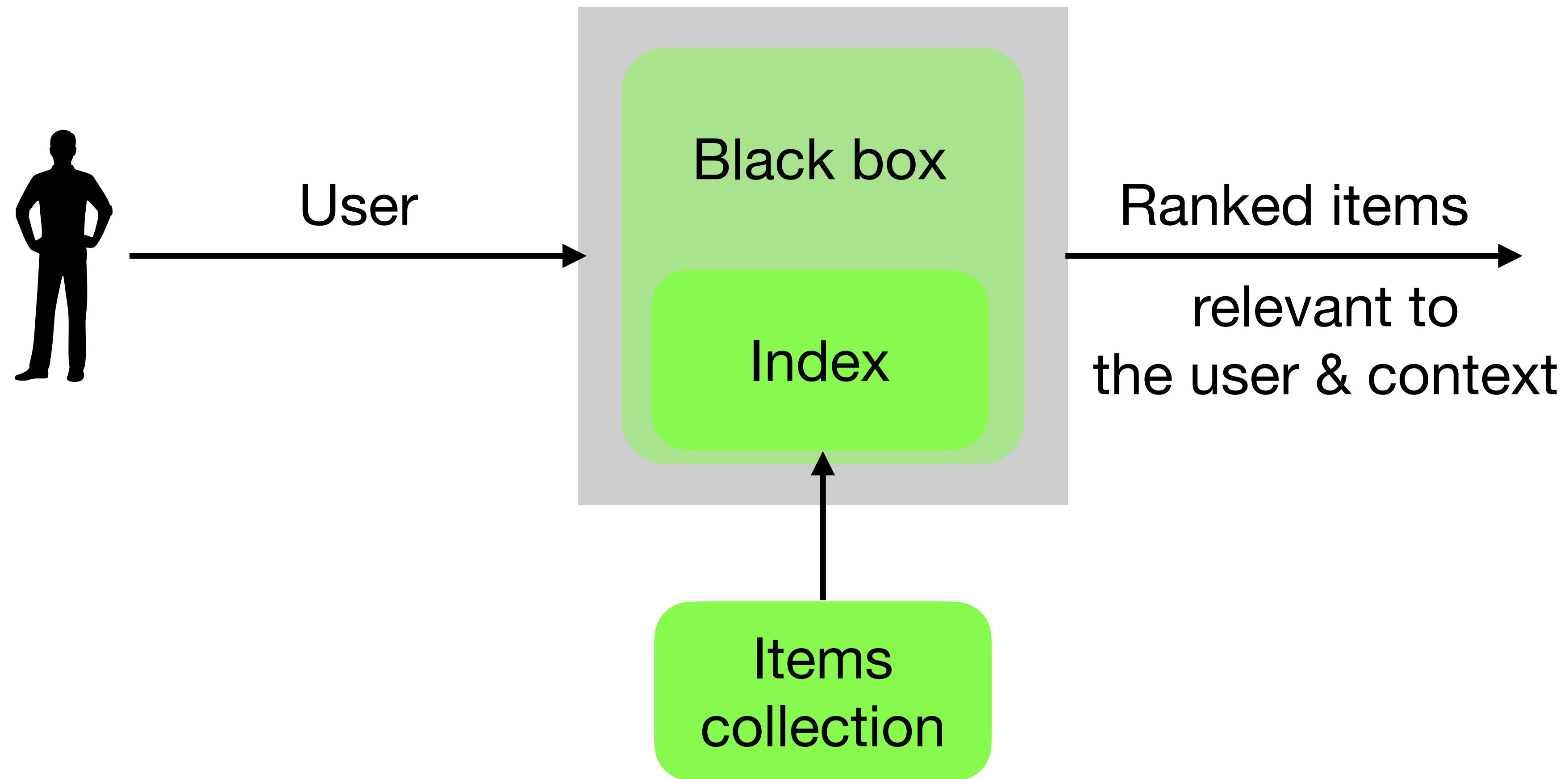
  uint64 client_id = 1;
  string session_id = 2;
  google.protobuf.Timestamp timestamp = 3;
  string event_date = 4;
  uint32 shelf_id = 5;
  repeated TRecord records = 6;
  repeated int64 experiment_id = 7;
  string request = 8;
  repeated int32 featureIDs = 9 [packed=true];
  repeated float user_features = 10 [packed=true];
  repeated int32 user_feature_ids = 11 [packed=true];
  repeated TModel models = 12;
  repeated int32 empty_feature_ids = 13 [packed=true];
  bool is_stress_test = 15;
}
```

```
{
  "id": 15,
  "name": "f_diff_count_items_cart_items_per_order"
},
{
  "id": 16,
  "name": "f_diff_count_items_ordered_items_per_client_1year"
},
{
  "id": 17,
  "name": "f_diff_count_items_current_items_per_client_1year"
},
{
  "id": 18,
  "name": "f_diff_count_items_cart_items_per_client_1year"
},
{
  "id": 19,
  "name": "f_contain_in_ordered_items"
},
{
  "id": 20,
  "name": "f_contain_in_viewed_items"
},
{
  "id": 21,
  "name": "f_contain_in_favorite_items"
},
}
```

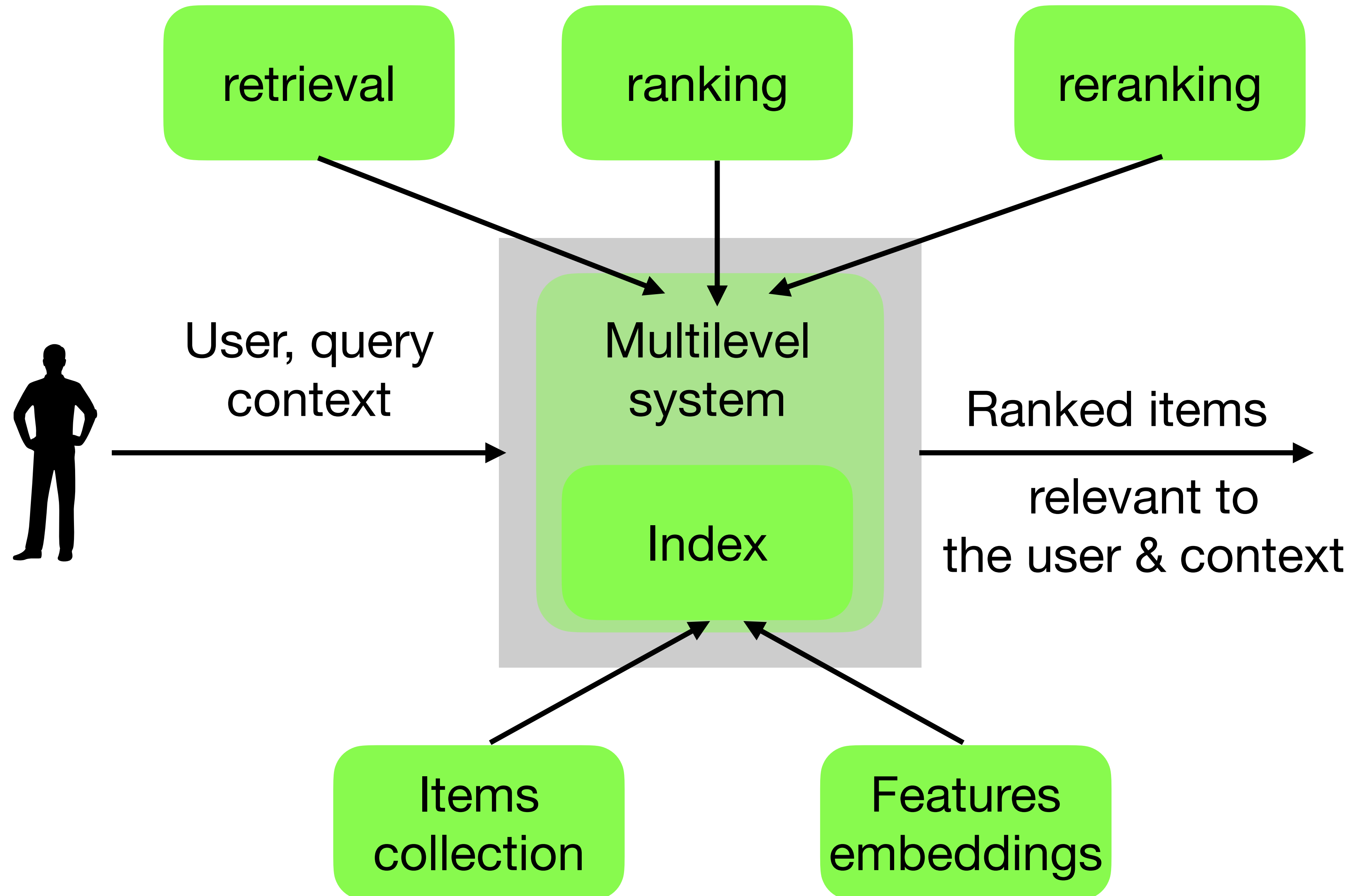
Многоуровневая система

Как это все учить?

Многоуровневая система



Многоуровневая система



Вопросы